# SafeLine WAF Home Lab Project

Protecting DVWA from Real Web Attacks (SQL Injection, XSS, HTTP Flood & More)

Ronak - Cybersecurity Home Lab
March 2026

## Project Overview

For this lab I wanted to get practical experience with both attacking and defending a web application.

I deployed DVWA (Damn Vulnerable Web Application) on an Ubuntu server and used a Kali Linux VM to perform several common web attacks such as SQL injection, cross-site scripting (XSS), and HTTP flood attempts.

Once I confirmed the attacks worked against the vulnerable application, I installed SafeLine WAF, an open-source Web Application Firewall, and placed it in front of the application to see how it detects and blocks malicious traffic.

The goal of the lab was to:

- Trigger real vulnerabilities such as SQL Injection, XSS, and HTTP flooding
- Observe how the attacks behave when no protection is in place
- Deploy SafeLine WAF and configure security rules
- Verify that the WAF detects and blocks malicious requests
- Review security events and logs generated by the WAF

This kind of traffic monitoring and alert investigation is similar to what SOC analysts deal with when monitoring web infrastructure.

## Lab Environment

I built this lab using virtualization so everything stayed isolated.

- Host machine: MacBook M4 Pro running Parallels Desktop
- Victim VM: Ubuntu Server 22.04 LTS (bridged networking, IP 10.0.0.61)
- Attacker VM: Kali Linux (bridged networking)
- Vulnerable app: DVWA (Apache + MariaDB running on port 8080)
- WAF: SafeLine (Docker-based, free + 7-day Pro trial)
- Protected URL: https://dvwa-lab (resolved using /etc/hosts)
- SafeLine GUI: https://10.0.0.61:9443/sites

Both machines were on the same bridged network, which allowed the Kali VM to directly interact with the Ubuntu server like a real attacker.

# Step-by-Step Walkthrough

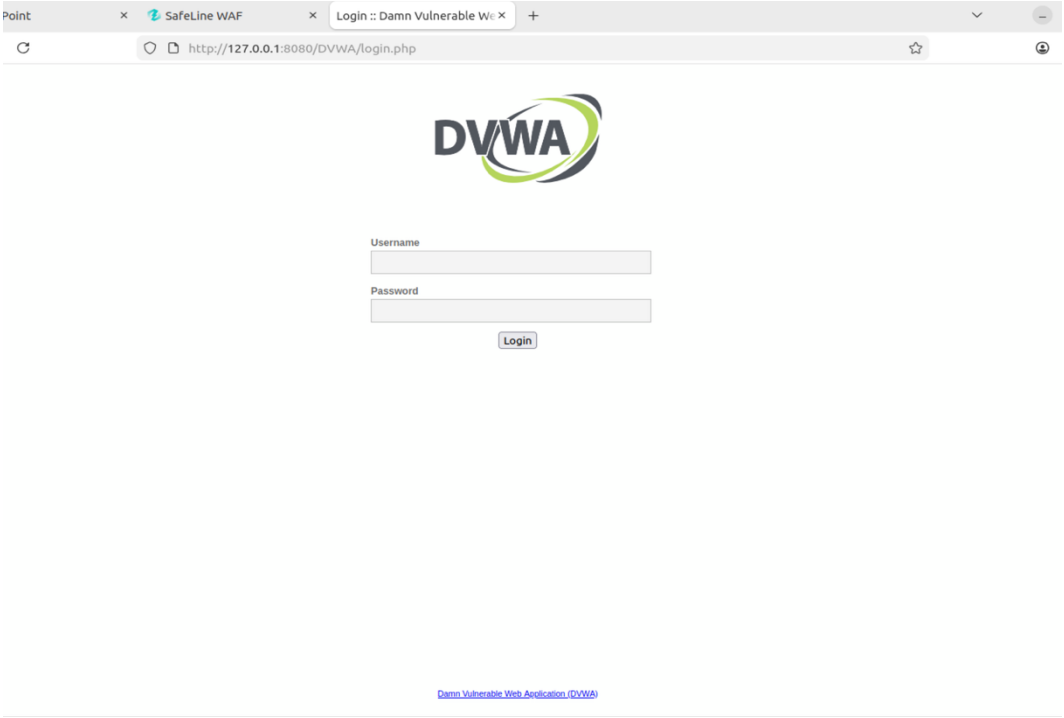
## 1. DVWA Deployment

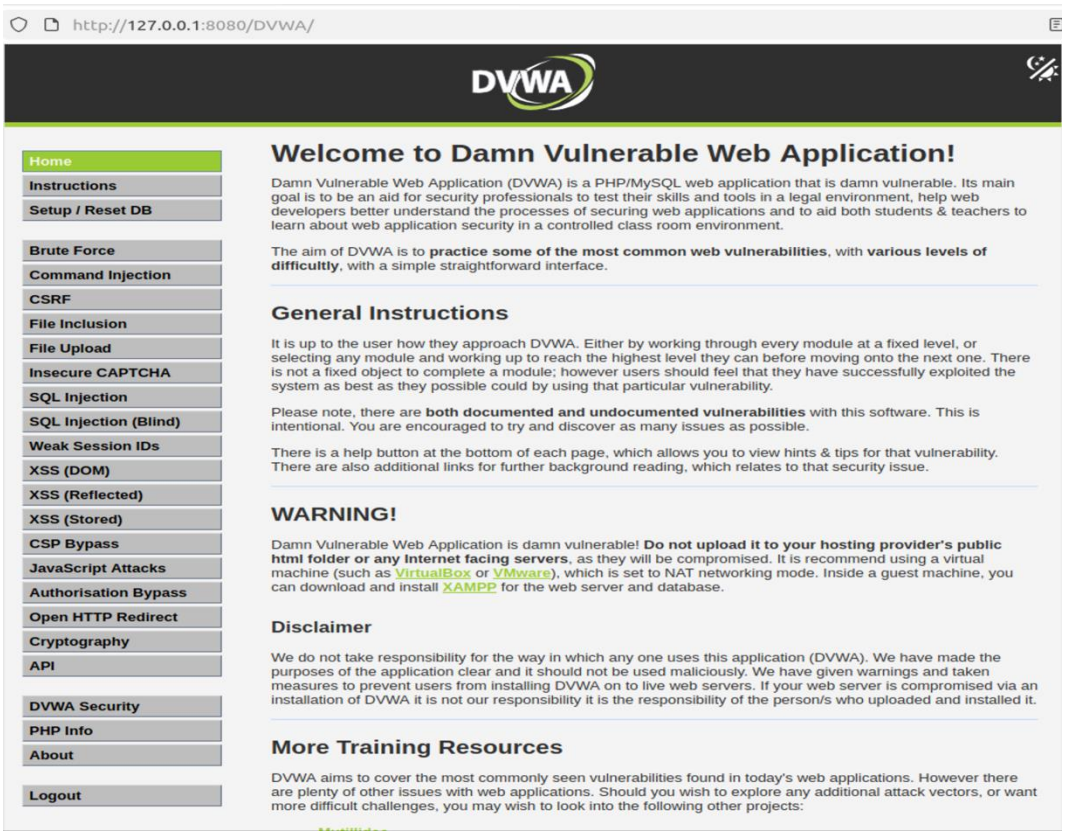
I started by installing a LAMP stack (Apache, MariaDB, and PHP) on the Ubuntu server and then cloned the DVWA repository.

During setup, I ran into a few database issues such as access denied and unknown database errors. After troubleshooting the MariaDB configuration and fixing user permissions, I was able to get DVWA connected properly.

I reset the database using the setup.php page and confirmed that the application was working.

For testing purposes, I set the DVWA security level to Low, which makes it easier to demonstrate vulnerabilities.


Point
SafeLine WAF
Login :: Damn Vulnerable We
http://127.0.0.1:8080/DVWA/login.php
DVWA
Username
Password
Login
Damn Vulnerable Web Application (DVWA)


http://127.0.0.1:8080/DVWA/
DVWA
Home
Instructions
Setup / Reset DB
Brute Force
Command Injection
CSRF
File Inclusion
File Upload
Insecure CAPTCHA
SQL Injection
SQL Injection (Blind)
Weak Session IDs
XSS (DOM)
XSS (Reflected)
XSS (Stored)
CSP Bypass
JavaScript Attacks
Authorisation Bypass
Open HTTP Redirect
Cryptography
API
DVWA Security
PHP Info
About
Logout
Welcome to Damn Vulnerable Web Application!
Damn Vulnerable Web Application (DVWA) is a PHP/MySQL web application that is damn vulnerable. Its main goal is to be an aid for security professionals to test their skills and tools in a legal environment, help web developers better understand the processes of securing web applications and to aid both students & teachers to learn about web application security in a controlled class room environment.
The aim of DVWA is to practice some of the most common web vulnerabilities, with various levels of difficultly, with a simple straightforward interface.
General Instructions
It is up to the user how they approach DVWA. Either by working through every module at a fixed level, or selecting any module and working up to reach the highest level they can before moving onto the next one. There is not a fixed object to complete a module; however users should feel that they have successfully exploited the system as best as they possible could by using that particular vulnerability.
Please note, there are both documented and undocumented vulnerabilities with this software. This is intentional. You are encouraged to try and discover as many issues as possible.
There is a help button at the bottom of each page, which allows you to view hints & tips for that vulnerability. There are also additional links for further background reading, which relates to that security issue.
WARNING!
Damn Vulnerable Web Application is damn vulnerable! Do not upload it to your hosting provider's public html folder or any Internet facing servers, as they will be compromised. It is recommend using a virtual machine (such as VirtualBox or VMware), which is set to NAT networking mode. Inside a guest machine, you can download and install XAMPP for the web server and database.
Disclaimer
We do not take responsibility for the way in which any one uses this application (DVWA). We have made the purposes of the application clear and it should not be used maliciously. We have given warnings and taken measures to prevent users from installing DVWA on to live web servers. If your web server is compromised via an installation of DVWA it is not our responsibility it is the responsibility of the person/s who uploaded and installed it.
More Training Resources
DVWA aims to cover the most commonly seen vulnerabilities found in today's web applications. However there are plenty of other issues with web applications. Should you wish to explore any additional attack vectors, or want more difficult challenges, you may wish to look into the following other projects:

## 2. SafeLine WAF Installation

Next, I installed SafeLine WAF using the official installation script:

Bash

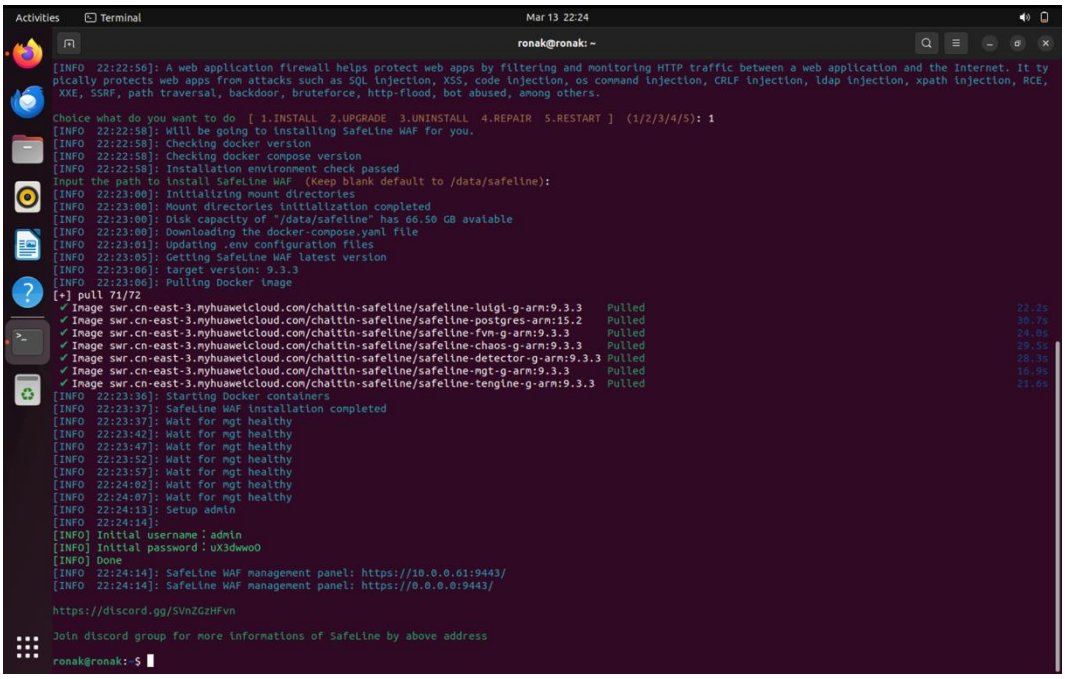
```
sudo bash -c "$(curl -fsSLk
https://waf.chaitin.com/release/latest/manager.sh)" -- --en
```

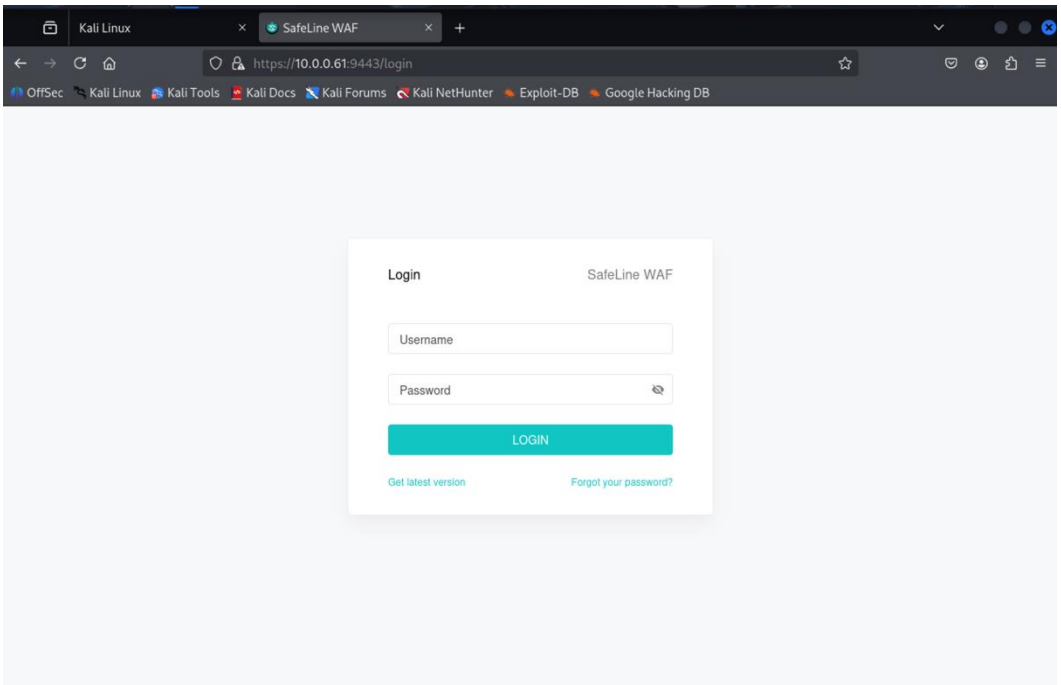
The installer automatically set up Docker and deployed all required containers.

Once the installation finished, I accessed the SafeLine dashboard at:

https://10.0.0.61:9443/sites

I logged in, changed the default password, and activated the 7-day Pro trial to test additional features.






## 3. Self-Signed SSL Certificate

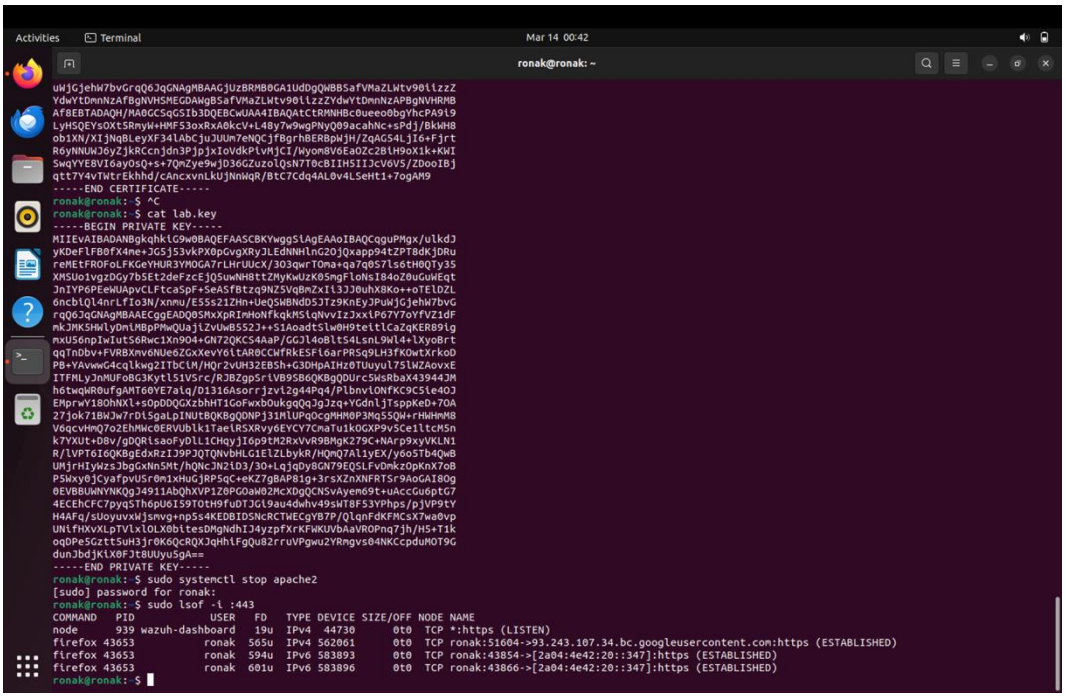
To enable HTTPS, I generated a self-signed SSL certificate using OpenSSL.

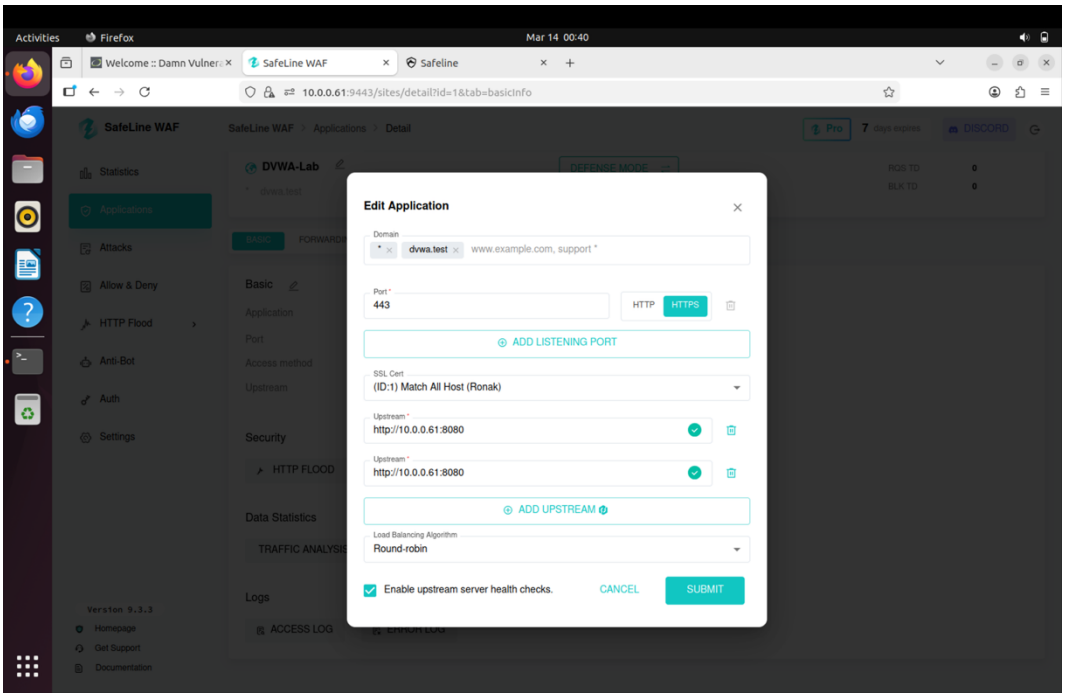
Bash

```
openssl genrsa -out priv.key 4096
openssl req -new -key priv.key -out priv.csr
openssl x509 -req -days 365 -in priv.csr -signkey priv.key -out priv.crt
```

After generating the certificate files, I uploaded them in the SafeLine dashboard under SSL Certificates and assigned the certificate to the DVWA application.

This allowed the application to run securely over HTTPS through the WAF.


Activities
Terminal
Mar 14 00:42
ronak@ronak: ~
-----END CERTIFICATE-----
ronak@ronak:~$ ^C
ronak@ronak:~$ cat lab.key
-----BEGIN PRIVATE KEY-----
-----END PRIVATE KEY-----
ronak@ronak:~$ sudo systemctl stop apache2
[sudo] password for ronak:
ronak@ronak:~$ sudo lsof -i :443
COMMAND PID USER FD TYPE DEVICE SIZE/OFF NODE NAME
node 939 wazuh-dashboard 19u IPv4 44730 0t0 TCP *:https (LISTEN)
firefox 43653 ronak 565u IPv4 562061 0t0 TCP ronak:51604->93.243.107.34.bc.googleusercontent.com:https (ESTABLISHED)
firefox 43653 ronak 594u IPv6 583893 0t0 TCP ronak:43854->[2a04:4e42:20::347]:https (ESTABLISHED)
firefox 43653 ronak 601u IPv6 583896 0t0 TCP ronak:43866->[2a04:4e42:20::347]:https (ESTABLISHED)
ronak@ronak:~$


Activities
Firefox
Mar 14 00:40
Welcome :: Damn Vulner
SafeLine WAF
Safeline
10.0.0.61:9443/sites/detail?id=1&tab=basicInfo
SafeLine WAF
SafeLine WAF > Applications > Detail
Statistics
Applications
Attacks
Allow & Deny
HTTP Flood
Anti-Bot
Auth
Settings
DVWA-Lab
Basic
Application
Port
Access method
Upstream
Security
HTTP FLOOD
Data Statistics
Logs
ACCESS LOG
Edit Application
Domain
dvwa.test
www.example.com, support *
Port *
443
HTTP
HTTPS
ADD LISTENING PORT
SSL Cert
(ID:1) Match All Host (Ronak)
Upstream *
http://10.0.0.61:8080
Upstream *
http://10.0.0.61:8080
ADD UPSTREAM
Load Balancing Algorithm
Round-robin
Enable upstream server health checks.
CANCEL
SUBMIT
Version 9.3.3
Homepage
Get Support
Documentation

## 4. Protected DVWA Application Setup

I then configured DVWA inside SafeLine.

In the dashboard: Applications → Add Application

Configuration:

- Domain: dvwa-lab
- Backend server: http://127.0.0.1:8080 (or http://10.0.0.61:8080)
- Frontend protocol: HTTPS

Since this is a local lab, I added entries to the /etc/hosts file:

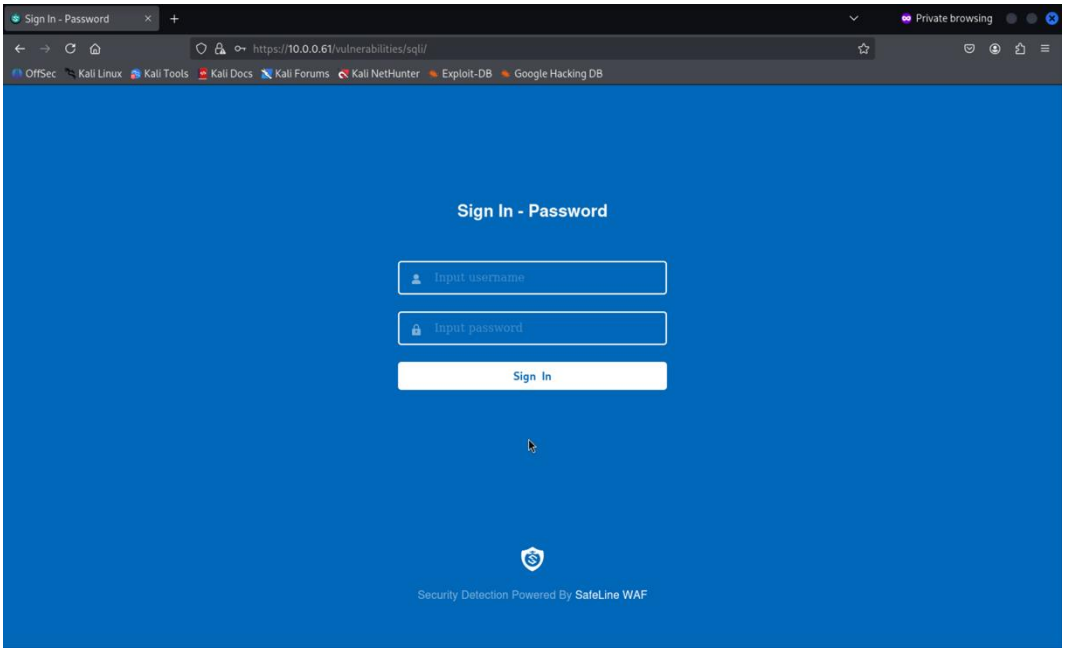
Ubuntu VM: 127.0.0.1 dvwa-lab

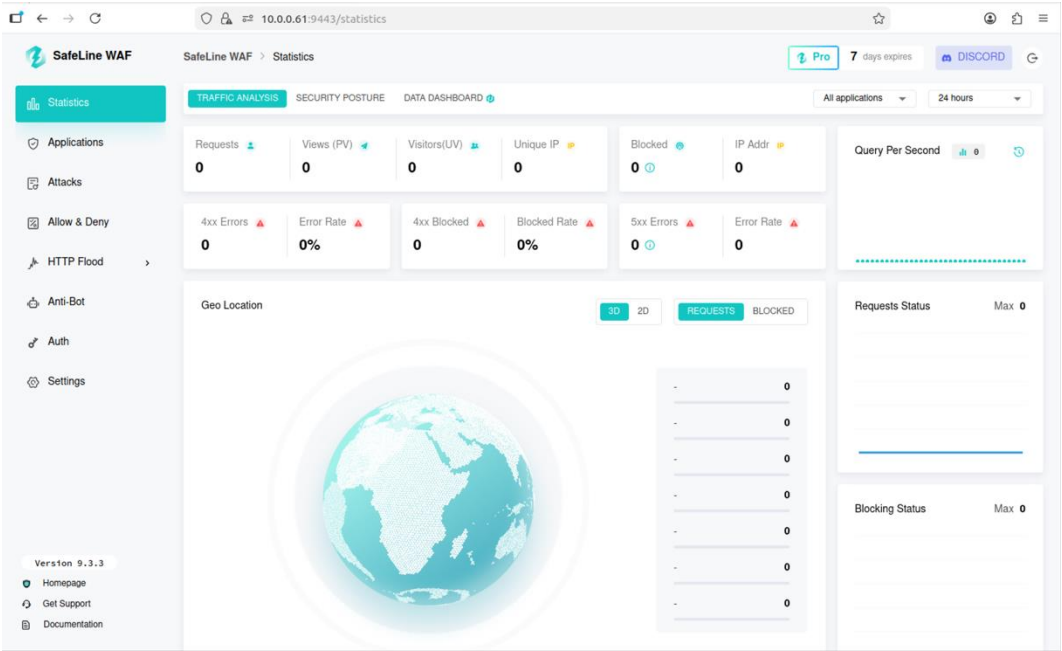
Kali VM: 10.0.0.61 dvwa-lab

After this setup, DVWA was accessible through:

https://dvwa-lab or https://10.0.0.61

At this point, SafeLine was acting as a reverse proxy, inspecting traffic before it reached the application.



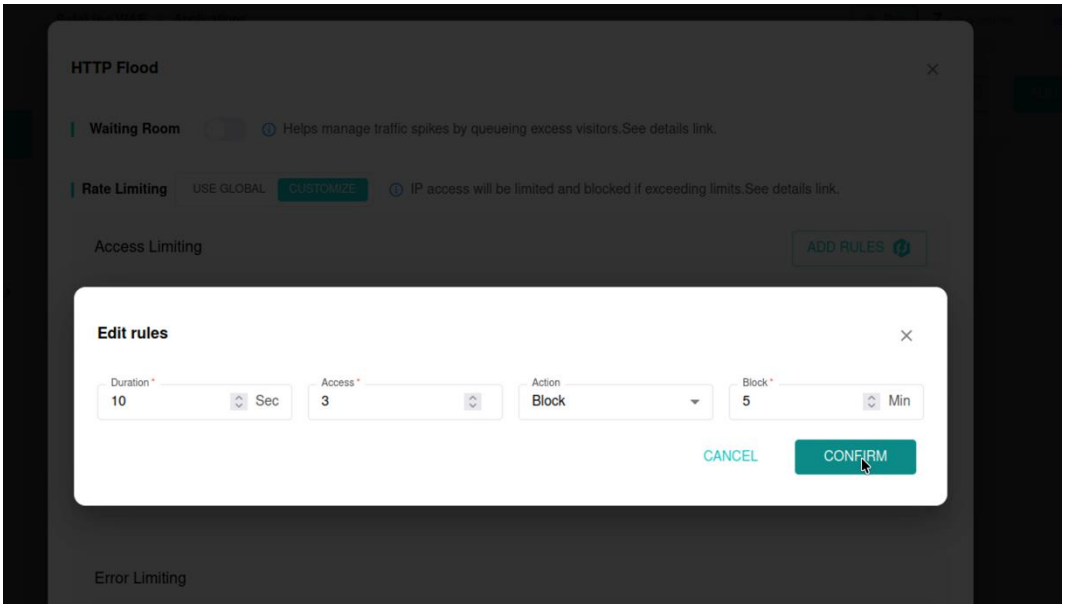


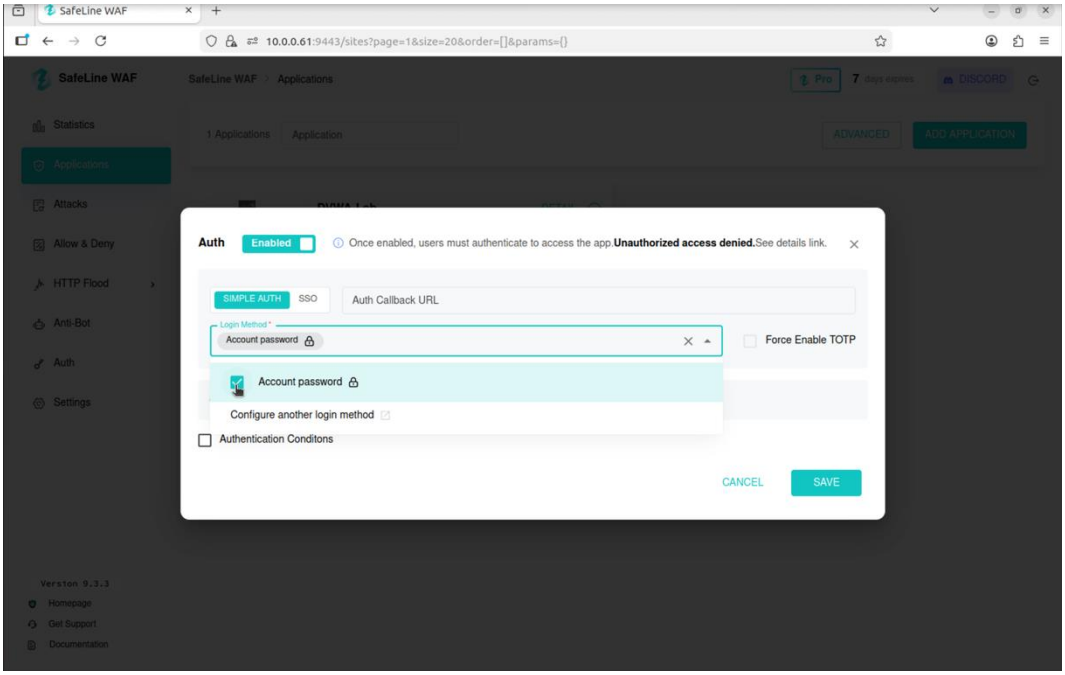

## 5. **Configured Security Rules**

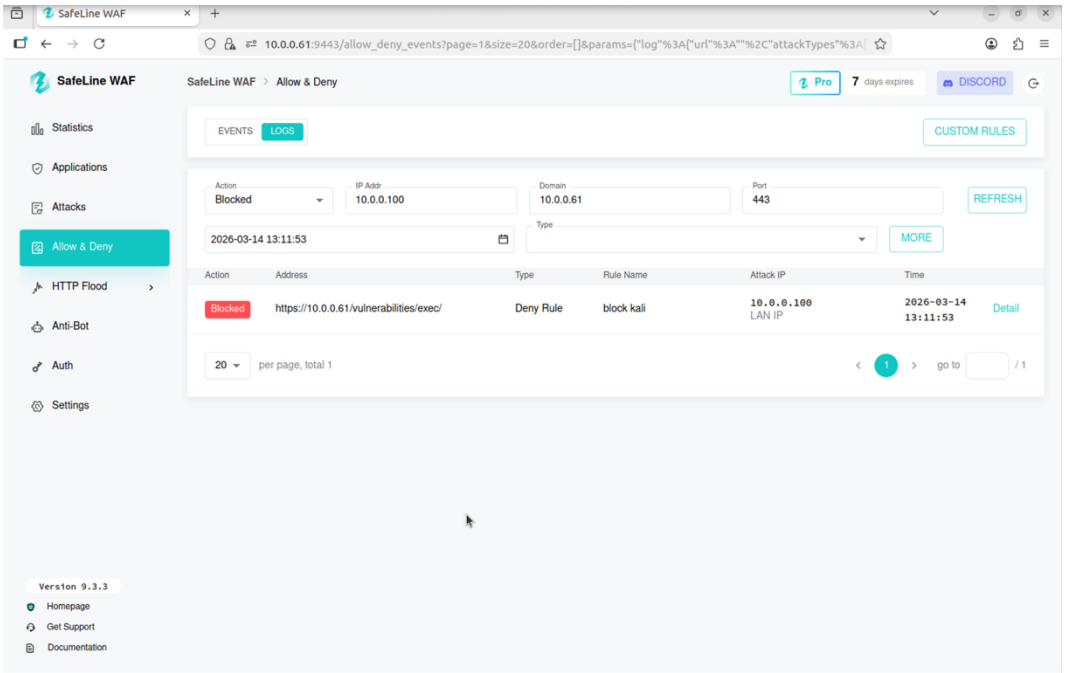
I enabled and tuned several protections in SafeLine.

- **HTTP Flood Defense:** Block after 3 requests within 10 seconds (IP blocked for 5 minutes)
- **Custom Authentication**: Added a username/password requirement and enforced it for the attacker IP
- **Custom Deny Rule:** Created a rule to immediately block the Kali machine IP for testing
- **SQL Injection & XSS Protection:** Enabled using Balance mode

These rules helped simulate how different types of malicious traffic can be detected and controlled.


HTTP Flood
Waiting Room
Helps manage traffic spikes by queueing excess visitors.See details link.
Rate Limiting
USE GLOBAL
CUSTOMIZE
IP access will be limited and blocked if exceeding limits.See details link.
Access Limiting
ADD RULES
Edit rules
Duration *
10
Sec
Access *
3
Action
Block
Block *
5
Min
CANCEL
CONFIRM
Error Limiting


SafeLine WAF
10.0.0.61:9443/sites?page=1&size=20&order=[]&params={}
SafeLine WAF
SafeLine WAF > Applications
Pro
7 days expires
DISCORD
Statistics
Applications
Attacks
Allow & Deny
HTTP Flood
Anti-Bot
Auth
Settings
1 Applications
Application
ADVANCED
ADD APPLICATION
Auth
Enabled
Once enabled, users must authenticate to access the app.Unauthorized access denied.See details link.
SIMPLE AUTH
SSO
Auth Callback URL
Login Method *
Account password
Force Enable TOTP
Account password
Configure another login method
Authentication Conditons
CANCEL
SAVE
Version 9.3.1
Homepage
Get Support
Documentation

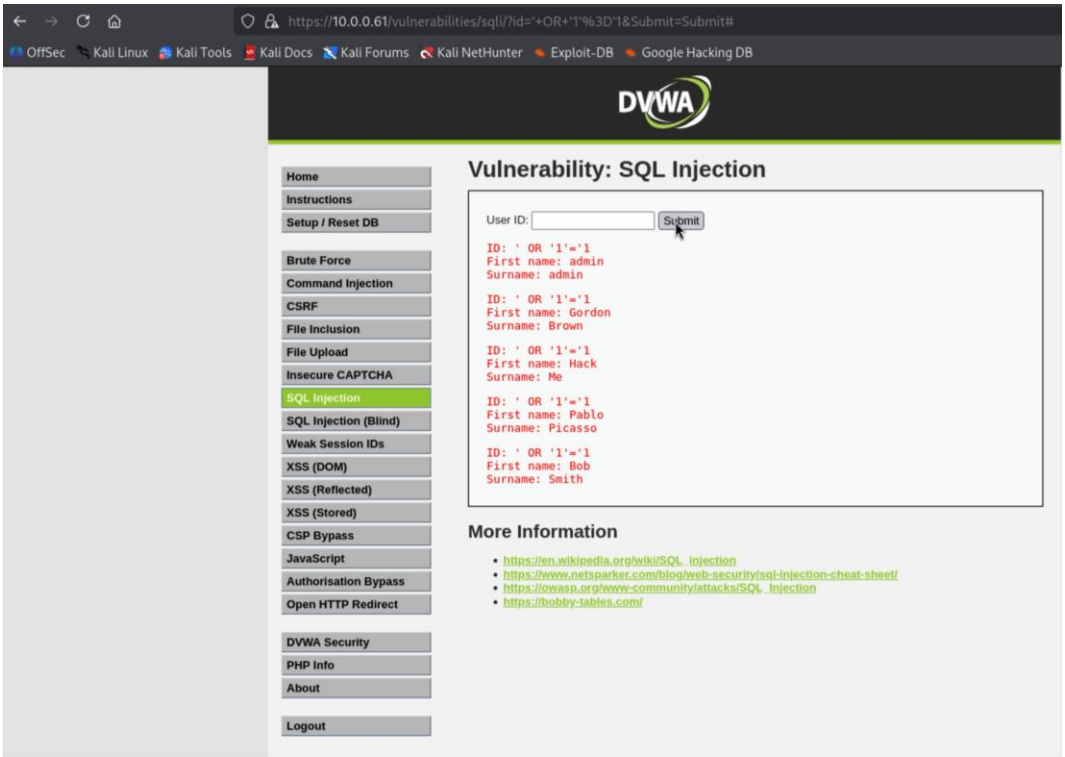



## 6. **Attack Simulation & Blocking**

From the Kali Machine, I targeted: **https://dvwa-lab** (or https://10.0.0.61).

**Without protection (baseline):**

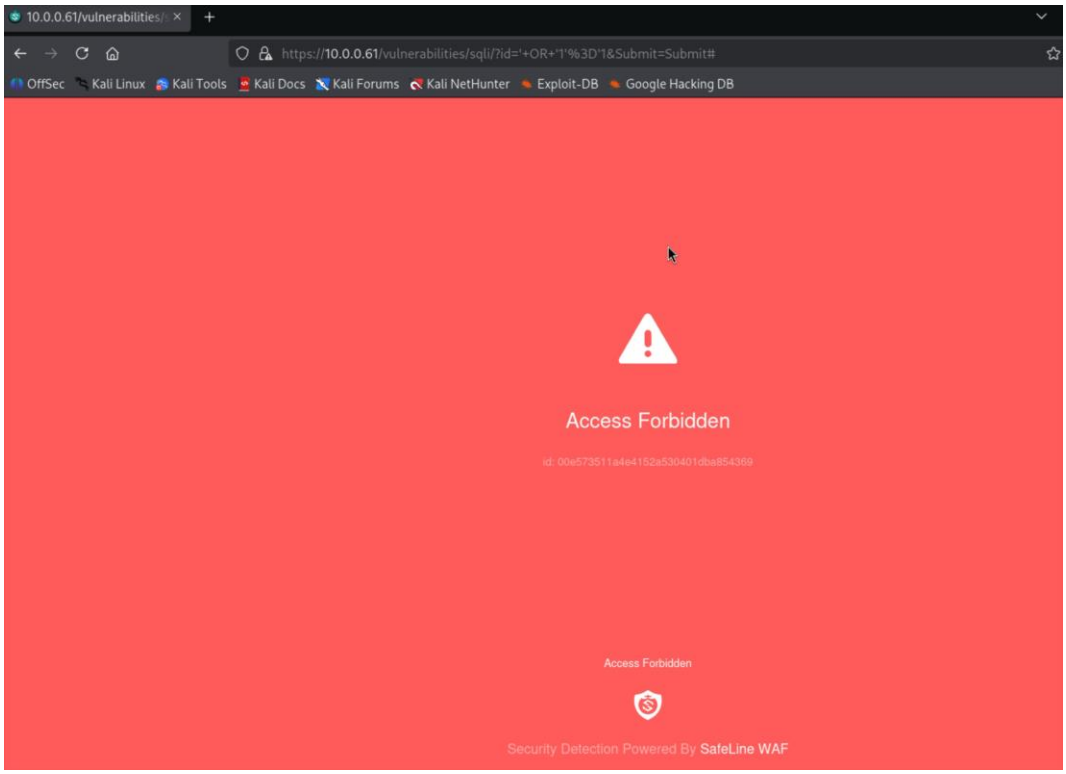
- SQL Injection: `' OR '1'='1` → successfully returned all users




**With WAF protections enabled:**

- Same SQL Injection → blocked (403 response + log entry)
- Reflected XSS → blocked
- HTTP Flood (rapid requests) → IP temporarily blocked
- Custom deny rule → immediate block

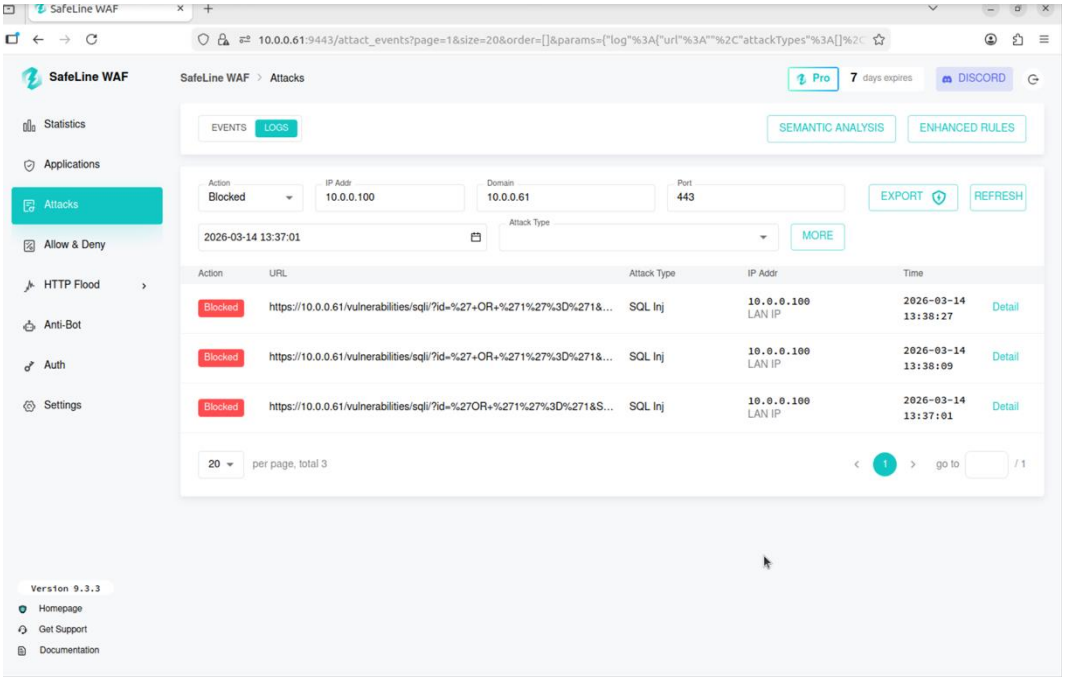
What stood out to me was how quickly SafeLine detected the SQL injection payload. The request was blocked almost instantly, and a log entry was generated showing the matched rule and attacker IP.



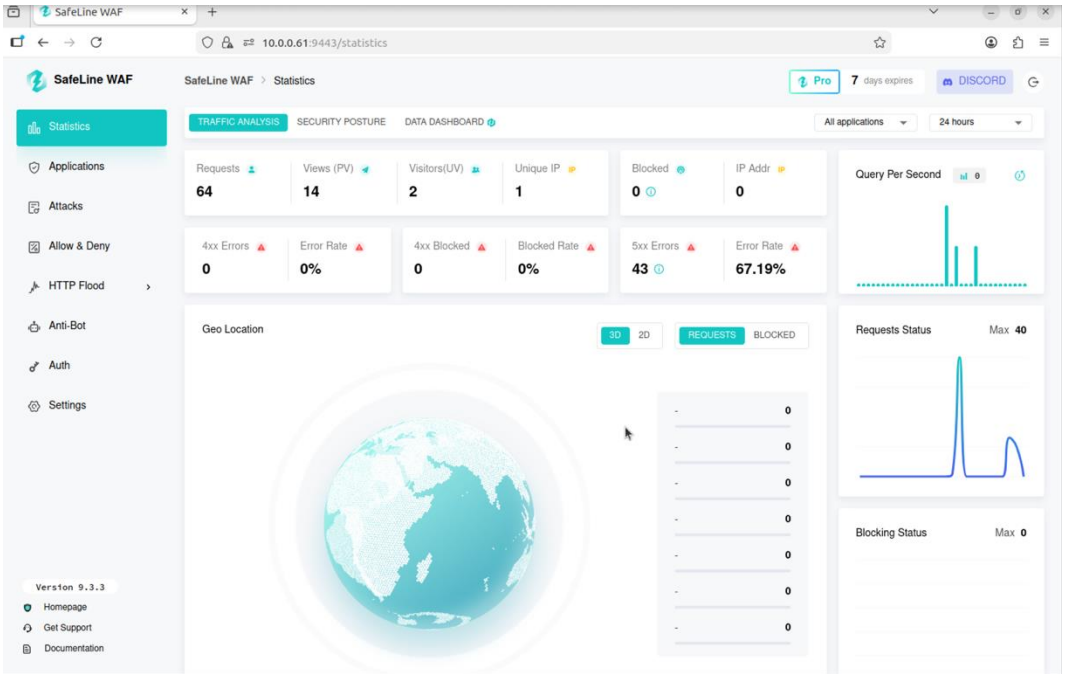

All events appeared live in the SafeLine dashboard with details such as:

- Source IP
- Payload
- Matched rule
- Timestamp
- Action taken

These logs are very similar to the types of alerts SOC analysts review during investigations.




After the request was blocked, I checked the SafeLine dashboard to review the generated logs.




## Key Learnings & SOC Relevance

This lab helped me connect theory with practical experience.

- I saw how real web attacks like SQL injection and XSS are executed
- Learned how a WAF works as a reverse proxy to inspect traffic
- Configured security rules like rate limiting, IP blocking, and authentication
- Analyzed logs generated when malicious traffic was blocked
- Gained experience troubleshooting Linux services and networking issues

This type of activity closely matches what SOC analysts do when investigating suspicious web traffic, such as blocked injection attempts or abnormal request patterns.

## Resources Used

- Tutorial: “EASY CYBERSECURITY Home Lab – SafeLine WAF” by The Social Dork
- SafeLine documentation and installation guide
- DVWA GitHub repo
- Full screenshots & recordings: Github